물고기 로봇 운용 설명서 — 전원 투입부터 종료까지

전원을 켜고, 확인하고, 보정하고, 물에 넣고, 꺼내서 정리하기까지의 전 절차.

로그 문구는 전부 **실제 기동 기록**에서 그대로 옮긴 것이다 (최종 대조 2026-08-26).

최근 개정 2026-10-01: **AUTO 진입은 당분간 무조건 `static`**(완전 정지 측정, MANUAL 로 나올 때까지). AUTO 진입 후 **10초 대기**(물속 영점 확보용)·3초 주행·추력 40 %, 뱅크 선회(bank_left/bank_right)·완전 정지(static) 시나리오, 7장 `ros2 param` 신설,

실험 전 복각 점검 `dip_check.py` 신설 (D.3).

설계 근거와 측정 이력은 `CLAUDE.md`와 `docs/imu_detailed.md`, 남은 작업은 `docs/todo.md`.

표기: 🎮 = 조종기로 한다 · 💻 = 터미널로 한다 · 👀 = 눈으로 확인한다

---

0. 한눈에 — 조종기 지도

이 로봇의 일상 운용은 **버튼 두 개**로 끝난다. 이 표가 설명서의 중심이다.

	짧게 (0.04~1초, "톡")	**3초 (정확히 3초에 1회 발동)**
btn1	**AUTO / MANUAL 전환** (AUTO 는 **10초 뒤 시작**. 현재 기본은 `static` — 끝까지 측정, btn1 으로 종료)	**대기압 영점 재포착** — 공기 중 전용, 잠기면 거부
btn2	**현재 방향을 yaw 0°로** (헤딩 영점)	**지자기 보정 시작** (텀블 필요; 한 번 더 3초 = 조기 종료)
btn1+btn2 동시	(아무 동작 없음 — 단독 동작만 억제)	**3초 유지 = AUTO 시나리오 전환**. 현재 목록은 `static` 하나라 **눌러도 안 바뀐다** (7.3 에서 되돌리는 법)

- **1초를 넘기고 3초가 안 되면 아무 일도 없다** — 짧지도 길지도 않은 구간은 무시된다.
- 3초는 `== 300틱` 판정이라 계속 눌러도 한 번만 발동하고, 손을 뗄 때 짧은 누름으로 세지 않는다.

스틱: 좌우 스틱이 roll/pitch **목표 자세**(±25.5°)와 스로틀이다. **요 스틱은 쓰지 않는다** —

2026-09-14 부터 요 제어가 없고 요 서보는 중앙(1500)에 고정이다. 스틱은 서보를

직접 미는 게 아니라 "이 자세로 가라"는 목표를 주고, PID가 그 목표를 추적한다.

시스템 구성 요약: Pi 5(추정·제어·기록, ROS2 노드 8개) ↔ UART ↔ nRF52840(조종기 링크,

IMU, 서보/ESC PWM). 압력 센서 3개(노즈 정체압 + 좌우 정압)로 수심·속도를 잰다.

---

1. 전원 켜기

1.1 켜기 전

1. **로봇을 고정한다.** 전원이 들어오면 서보가 움직일 수 있다.
2. **흔들리지 않는 곳에 둔다.** 기동 후 약 10초간 자이로 영점을 잡는다.
3. **물이 없어야 한다.** 대기압 영점을 공기 중에서 잡는다. 물속에서 잡히면 그 깊이가

"수면"이 되어 수심이 통째로 틀어진다.

4. 🎮 조종기는 **로봇보다 먼저** 켜고, **스로틀 스틱은 최하**에 둔다.

1.2 타임라인 (t = 서비스 시작 시각)

시각	일어나는 일	사람이 할 일
t + 0초	systemd가 `fish-robot.service` 시작	—
t + 45초	ROS2 launch, 노드 8개 기동	—
t + 47초	EKF 초기화(50샘플) → **자세 발행 시작** · 복각 1회 검사	—
t + 47~57초	**자이로 영점 정지 창 (10초)**	👀 **가만히 둘 것**
t + 47초	`[Hydro] 대기압 영점 웜업 대기 (0 / 180초)`	—
t + 225초	대기압 영점 수집(101샘플, 9초)	👀 건드리지 말 것
t + 234초	영점 완료 → **수심 유효**	물에 넣어도 됨

전원 인가부터 수심이 나오기까지 대략 4분. 단 **속도는 t=0부터 나온다** —

속도(차압)는 대기압 영점이 필요 없고, 직전 세션의 q 영점(하루 유효)을 자동 적재한다.

자이로 정지 창이 실패하면 (움직였을 때) 창을 다시 시작한다, 최대 3회:

```
[WARN] [EKF] 정지 판정 실패 (자이로 std=0.80도/초, |a| 변동=0.223 g) -> 창 재시작 (1/3)
```

4번째에도 실패하면 불확실성을 10배로 부풀려 수용하고 ERROR를 남긴다 — 어쨌든 진행은 된다.

이 10초는 자이로 바이어스·중력 기준·지자기 크기 기준 **세 가지를 한 번에** 확정하므로

움직이면 셋이 같이 오염된다.

1.3 ⚠ 빌드했으면 반드시 재시작 — 실행-디스크 격차

서비스가 도는 중에 `colcon build`를 하면 **도는 프로세스는 옛 바이너리 그대로**다.

이 함정에 두 번 걸렸다 (8/21에 나흘, 8/26에도 한 번). 확인법:

```
systemctl show fish-robot -p ActiveEnterTimestamp --value        # 서비스 기동 시각
```

```
stat -c %y ~/ros2_ws/install/fish_robot_pkg/lib/fish_robot_pkg/pid_control_node
# 바이너리가 기동 시각보다 새로우면 → sudo systemctl restart fish-robot
```

> 게인·서보 방향·AUTO 시나리오 값만 바꿀 거면 **빌드도 재시작도 필요 없다** — 7장 (`ros2 param`).

---

2. 기동 로그 확인 (💻 1분)

```
journalctl -u fish-robot -b | head -80      # 또는 --since "-5min"
```

✅ 정상일 때 (2026-08-26 실기록)

```
[INFO] [MPU9250] 6축 감시 켜짐 (±4 g / ±500 도/초, DLPF 41 Hz, 100 Hz 발행 /raw/imu_mpu9250, nRF IMU 칩 축으로 회전).
← 2026-09-16
[INFO] [AK8963] 지자기 센서 정상 연결 확인 (100Hz).
[INFO] [MS5837] 채널 0 (J5 커넥터) 압력센서 롬 로드 완료.        ← 0,1,2 전부
[INFO] [포트] front(ch0) 확인
[INFO] [포트] left(ch1) 확인
[INFO] [포트] right(ch2) 확인                                    ← 센서 지문 대조 3채널
[INFO] [EKF]   지자기 보정: bias(-7.453 20.538 -83.754) scale(1.0617 0.9660 0.9776)
[INFO] [EKF] 복각 기준 확보: 140.4도 (기대 143.0도, 편차 2.6도)
[INFO] [Hydro]   포트 배정: 정체압=ch0, 정압좌=ch1, 정압우=ch2  (파일, 물리확인 완료)
[INFO] [Hydro] 부호 검증: pitch=+10도(기수 아래), roll=0 -> dP_hydro=+2.988 mbar
[INFO] [PID Control] 게인: roll 8.75/0.00/0.00  pitch 8.75/0.00/0.00 (kp/ki/kd)
[INFO] [PID Control] 요 제어 없음: 요 서보 1500 고정, 요 스틱·시나리오 요 목표 무시 (2026-09-14)
[INFO] [PID Control] 서보 반전: 좌 0 / 우 1  스틱 데드밴드: rp 15, 스로틀 40
...180초 후...
[INFO] [Hydro] 대기압 영점 포착 완료 (3채널)
[INFO] [Hydro] q 영점을 공기 Δb 로 채웠습니다: 0.8250 mbar (...)  ← 속도 준비 완료
```

❌ 이것이 보이면 멈추고 조치

복각 오류 — 방위를 믿을 수 없다는 뜻. 이 한 줄이 방위 신뢰성의 유일한 판정이다.

```
[ERROR] [EKF] 복각이 물리적으로 불가능합니다: 실측 11.1도, 기대 143.0도
[ERROR] [EKF]   원인 후보: (1) 지자기 축 재배치의 Z 부호, (2) 하드아이언 보정값.
```

지자기 보정이 없거나 잘못됐다는 뜻 → **3.2의 지자기 보정을 하라.** 주의: 이 오류가 떠

있어도 `EKF_Flags`는 7을 유지하고 로봇은 그대로 돈다 — 수동 조종은 멀쩡해 *보이고*

AUTO의 절대 방위만 조용히 틀어진다. 실제로 8/25까지 보정 파일이 사라진 채 이 오류를

매 부팅 찍으며 돌고 있었다 (`imu_detailed.md` §10.1). 참고: 복각 기준값은 로봇이 놓인

자리의 자기 환경을 타므로, 철제 책상 근처에서는 140°대신 170°대가 잡히기도 한다 —

그 자리에서 절대 방위가 수십 도 틀릴 수 있다는 뜻이니 **AUTO 방위 시험은 깨끗한 자리에서.**

포트 배정 불일치 — 센서가 바뀌었다는 뜻

```
[ERROR] [포트] left(ch1) 불일치 — 그 자리 센서가 바뀌었습니다.
```

압력 채널 사망 / I2C 버스 잠김

```
[WARN] [MS5837] 채널 N 압력센서 응답 없음 (Reset 실패).
[ERROR] [I2C Driver] ... I2C 버스 잠김 의심 ...
```

조종기 없음 — 벤치에서는 정상, 2초마다 반복: `[PID Control] 수동 모드 중 조종기 통신 두절`

---

3. 물에 넣기 전 — 보정과 확인

3.0 체크리스트 (요약)

- ☐ 🎮 **스로틀 스틱 최하** — 벤치 작업 내내 유지하는 습관
- ☒ 부품·하우징을 만졌다면 → **3.2 지자기 텀블** (안 만졌으면 불필요)
- 🎮 **btn2 짧게** — 출발 방향을 yaw 0°로 (선택 — 요 제어가 없어 로그 읽기용일 뿐이다)
- 💻 **복각 점검**: `python3 $S/dip_check.py` → ✅ **실험 진행 가능** 이 나올 것 (D.3).

서비스는 **쇠붙이·자석에서 떨어진 자리에서** 켠다 — 켜는 순간의 자리가 나침판 기준이 된다

- ☒ 대기압 영점 — 켠 지 180초 지났으면 **자동으로 끝나 있다.** 하우징을 열었다 닫았거나

오래 켜뒀으면 🎮 **btn1 3초** 로 다시 (3.3)

- 👀 **서보 방향 확인** (3.4) — 피치 기울임 = 두 지느러미 **같은** 방향, 롤 = **반대** 방향
- ☒ 튜브·커넥터를 만졌다면 → 노즈에 입바람 불어 `data[0]`만 오르는지 (3.5)
- 💻 배터리 확인: `ros2 topic echo /rc/status --once` → [2],[3] ÷100 = V.

아무 노드도 전압을 감시하지 않는다 — 사람이 안 보면 어디에도 안 남는다

3.1 어떤 보정이 언제 필요한가

보정	언제	어떻게	어디서
자이로 영점	매 부팅 자동	켠 뒤 10초 정지	아무 데나
대기압 영점 (수심)	매 부팅 자동 (180초)	재영점은 btn1 3초	**공기 중만**
공기 Δb (속도 q 영점)	대기압 영점에 **자동 포함**	별도 조작 없음	공기 중
헤딩 영점	선택 (요 제어 없음, 로그용)	btn2 짧게	출발 방향 보고
지자기 (타원체)	**부품이 바뀌었을 때만**	btn2 3초 + 텀블	강철 1 m 밖
물속 q 영점	**선택** — 정지 속도 오차가 보일 때만	`/hydro/zero_q` 서비스	물속 15 cm+

대기창 물속 영점 (속도)	**매 AUTO 주행 자동** (2026-09-16)	별도 조작 없음 — 넣고 손 뗀 뒤 2초+ 가만히 (4.3)	물속, 정지

3.2 지자기 보정 — 텀블이 필요하다 (btn2 3초)

왜 움직여야 하나: 몸에 붙은 쇠붙이의 자기장 왜곡을 재려면 자기장 벡터가 자세에 따라

그리는 **타원체 궤적**을 봐야 한다. 한 자세로 가만히 있으면 점 하나뿐이라 수학적으로 불가능하다.

1. 강철에서 **1 m 이상** 떨어진 자리에서 🎮 btn2 **3초**
2. **한 자리에서** 6가지 자세(배 아래/배 위/좌로 눕힘/우로 눕힘/코 위/코 아래)를 하나씩

잡고, 그 자세 그대로 **제자리에서 한 바퀴**씩 돈다 (로봇을 들고 본인이 도는 게 쉽다)

3. 저널에 자세별 진행 막대가 흐른다. 6/6이 되면 자동 계산·저장·**즉시 적용** (재시작 불필요)
4. 조기 종료 = btn2 다시 3초

```
[WARN] [EKF] 지자기 캘리브레이션 중 -> 강제 6축 (완료 시 자동 복귀)
...텀블...
✅ 구형도 3.4% — 쓸 만합니다.  ✅ 복각 140도 — 축 재배치까지 맞다는 증거입니다.
[INFO] [EKF] 지자기 캘리브레이션 성공 — 새 보정 즉시 적용
```

안전장치: 흩어짐 8 % 초과면 저장을 거부하고 기존 파일을 지킨다.

평평한 스핀만 돌리면 안 된다 (수직 오프셋을 못 본다). **보정은 장소가 아니라 어셈블리에 묶인다** —

부품을 하나라도 바꾸면 무효 (실측: 조립 변경마다 하드아이언이 통째로 바뀌었다).

현재 보정: 2026-09-14 18:41, 흩어짐 5.0 %. 굴린 자리가 교란돼 텀블 복각이 168.5° 로 나왔다.

깨끗한 자리에서 켠 22:00 기동은 142.1° 였지만, **2026-09-15 에 자세를 바꿔 보니 복각이 자세마다 달랐다** —

똑바로 139°, 오른쪽으로 72° 눕힘 122~124°, 왼쪽으로 75° 눕힘 140~145°. 오른쪽으로 눕혀 가만히 둔 동안

EKF 가 나침판을 계속 버렸다. **좋은 보정은 모든 자세에서 몇 도 안에 든다 — 이 보정은 깨끗한 자리에서 다시 할 것.**

보정은 어디서 해도 되지만 그 자리는 깨끗해야 한다 — 결과의 `복각` 줄이 143° 근처인지 볼 것.

보정 뒤에는 `dip_check.py` 로 확인한다 (D.3).

> **텀블 후에는 btn1 3초로 대기압 재영점을 한다** — 로봇을 들고 돌면 고도 변화로
>
> 기압 영점이 오염된다 (1 m = 0.12 mbar). 순서: 텀블 → 내려놓고 → btn1 3초 → 9초 정지.

3.3 대기압 영점 (btn1 3초) — 공기 중 전용

켤 때는 버튼이 필요 없다 (180초 후 자동). btn1 3초는 다시 잡을 때만:

• 하우징을 열었다 닫았을 때 · 지자기 텀블 직후 · 오래 켜둬서 날씨로 기압이 변했을 때

(시간당 1~2 hPa = 수심 1~2 cm)

재영점은 웜업을 건너뛰고 **즉시 9초 수집**한다.

성공하면 **공기 Δb(속도 영점)도 자동으로 같이 채워진다** — 속도를 위한 별도 조작은 없다.

물에 잠긴 채 누르면 거부된다 (게이지압 5 mbar ≈ 5 cm 초과):

```
[ERROR] [Hydro] 물속으로 보입니다 (게이지압 29.30 mbar, 약 30 cm) — 대기압 영점을 잡지 않습니다.
        물 밖으로 꺼낸 뒤 다시 누르십시오. (정말 공기 중이면 /hydro/zero_atm 으로 강제)
```

흔들리거나 산포가 커도 거부된다. **어떤 경우든 거부·실패로 쓰던 영점이 사라지지 않는다** —

새 값이 성공해야만 교체되므로 수심이 도중에 끊길 일이 없다.

3.4 서보 방향 확인 (👀 30초)

MANUAL, 스틱 중립에서 로봇을 손으로 기울인다:

기울임	정상 반응 (물리)
코를 아래로	두 지느러미가 **같은 방향**으로, 코를 들어올리는 쪽으로 꺾임
옆으로 (롤)	두 지느러미가 **서로 반대**로, 기운 쪽을 들어올리는 쪽으로

우측 서보는 거울상 장착이라 소프트웨어가 반전한다(`servo_reverse_right` 기본 true).

방향이 반대로 보이는 채널은 재빌드 없이 즉석 반전:

```
ros2 param set /pid_control_node servo_reverse_left true    # 또는 right, true/false (요 서보는 중앙 고정이라 반전 파라미터 없음)
```

서보 조향각은 혼 기준 **±20°** (HS-5086WP, 1325~1675 µs), 오차 1° = 혼 1°, 20°에서 최대.

3.5 포트 신원 (튜브를 만졌을 때만)

노즈 포트에 입으로 바람 → `ros2 topic echo /sensor/pressure_raw` 에서 `data[0]`**만**

올라야 한다 (실측: 그 채널만 +3~24 mbar, 다른 채널 0.5 이하). 좌·우도 같은 방식.

확인 후 `config/port_map.txt`의 `verified:` 날짜를 갱신한다. 부팅 때마다 도는 PROM 지문

대조는 **센서 교체**만 잡고 **튜브 이동**은 못 잡는다 — 그래서 입바람이 필요하다.

---

4. 물속 운용

4.1 넣을 때

1. 대기압 영점이 끝났는지 확인 (수심 유효 — 1.2 타임라인의 t+234초, 또는 재영점 후)
2. **정체압(노즈) 포트의 기포를 뺀다** — 물속에서 로봇을 좌우로 흔들거나 노즈를 아래로

기울여 퍼지. 기포가 갇히면 속도가 오프셋을 갖고 굼떠진다. 첫 습식 시험의 가장 유력한

실패 요인이다

3. 💻 수심 확인: `ros2 topic echo /filtered/hydro` → `data[0]` = 수심[m]. 자로 대조

4.2 수심·속도 읽기

`/filtered/hydro` = `[수심 m, 속도 m/s, q, q_raw, dP_hydro, 정압 게이지, 사용한 pitch, 플래그]`

+ 뒤 7칸은 대기창 물속 영점 (2026-09-16: 영점, 출처, 구간 시작·끝 나이, 정지 판정, q_wz, speed_wz).

- **플래그 정상값 0x01F** (수심·속도 유효 + 두 영점 + 자세 신선). 비트별 뜻은 `docs/csv_format.md`
- **공기 중에서 속도가 −0.6 m/s쯤 나오는 것은 알려진 인공물** — dP_hydro가 물 밀도를

가정해서 그렇다. 물에 들어가면 사라진다. 공기 중 큰 음수 속도는 역으로 "물 밖" 신호다

- 물속 정지 상태에서 Speed가 0이 아니면 그 크기가 곧 기하 모델 오차다 — 크면(>0.1 m/s)

물속 q 영점을 잡는다: 로봇을 물속에 **정지**시키고

`ros2 service call /hydro/zero_q std_srvs/srv/Trigger` (3초 수집, 흐름·꼬리 정지 필수)

- **AUTO 주행의 속도는 `Speed`(41열) 대신 `Speed_wz`(54열)를 본다** (2026-09-16). 공기 중 영점은

젖으면 채널마다 흘러 정지 상태에서 +0.3~0.4 m/s 가 남는다(0913~0915 수조 로그). `Speed_wz` 는

AUTO 진입 대기 중에 잡은 **그 주행의 영점**을 뺀 값이다 — 4.3 "대기 중 물속 영점".

4.3 AUTO 모드 (🎮 btn1 짧게)

기본 시나리오(2026-09-16 주행 3초로, 09-15 4초·40 %, 종전 6초, 그 전 20 %·8초): **수평 자세로 추력 40 % (PWM 1400), 3초 전진 후 정지.**

자세는 전 구간 roll/pitch 0°를 지시하고(요는 제어하지 않는다), 3초가 지나면 추력만 1000으로 끊는다.

어떤 시나리오든 `run_sec` 이 지나면 추력을 끊고 멈춘다. **예외는 `static`** — 그것만

`run_sec` 을 무시하고 MANUAL 로 나올 때까지 계속한다 (2026-10-01).

AUTO 에 들어가면 10초 기다렸다가 시작한다 (2026-09-15 신설, 09-19 에 5 → 10초, `start_delay_sec`).

기다리는 동안은 추력 0 에 수평만 지시하고, 3초는 대기가 끝난 순간부터 센다. 대기 중 btn1 으로 MANUAL 에 나가면 취소되고, 다시 들어오면 처음부터 10초를 기다린다. **10초인 이유는 아래 물속 영점** — 로봇이 가라앉아 자리 잡을 시간(4~5초)에 정지 2초와 여유를 더한 값이다.

대기 중 물속 영점 (2026-09-16). 이 대기 동안 `hydro_estimator_node` 가 로봇이 조용한 구간의 동압을 그 주행의 속도 영점으로 잡는다 — 수심 변화 < 5 mm/s 이고 자이로 < 5 °/s 인 상태가 **2초 이상** 이어진 구간 중 가장 늦은 것의 중앙값. 그런 구간이 없으면 대기 마지막 2초로 폴백한다. 결과는 저널에 `[Hydro] 대기창 영점 확보: X mbar (정지 구간 …)` 또는 `(**폴백** …)` 으로 남고, CSV 49열 `Zero_src` 가 1(정지) / 2(폴백) / 0(없음) 이다.

- 버튼을 누른 뒤 손으로 넣는 구조라 **버튼 직후 4~5초는 가라앉는 중**이다(0915 실측 14~21 mm/s). 5초로는 정지 2초가 안 잡혀 폴백이 잦았다 — **그래서 2026-09-19 부터 기본 대기가 10초다.** 그래도 폴백(`Zero_src`=2)이 계속되면 `wet_zero_still_gyro_dps` 를 7.0 으로 완화한다 (7.3).
- 넣고 나서 **손을 떼고 2초 이상 가만히** 둔다. 잡고 있으면 그 손떨림이 영점에 들어간다.
- 주행마다 새로 잡는다 — 꺼냈다 넣으면 영점이 바뀌기 때문이다. 종전 `Speed`(공기 영점)도 그대로 기록되므로 둘을 비교할 수 있다 (`docs/csv_format.md` 48~55열).

다시 달리려면 btn1 으로 MANUAL 로 나갔다가 AUTO 로 재진입한다(진입 순간 스톱워치가 0으로 리셋된다). 종전의 6초 주기 피치 ±10° 왕복은 물속 첫 검증을 위해 걷어냈다 — 자세를 흔들면 추력 문제인지 자세 문제인지 가릴 수 없기 때문이다.

직진 시나리오는 실기로 확인됨 (2026-09-03, 운용자).

시나리오 목록과 교체

이름	동작
`straight` (기본)	수평 직진
`dive`	코를 `angle_deg` 만큼 **아래로** 하고 전진 (이 로봇은 양수 피치 = 코 아래)
`climb`	코를 `angle_deg` 만큼 위로 하고 전진
`porpoise`	3초마다 피치 ±10° 왕복하며 전진 (`run_sec` 을 늘려 쓸 것)
`attitude`	추력 0 으로 피치 ±10° 왕복 — 벤치에서 서보만 볼 때
`bank_left`	롤을 `angle_deg` 만큼 **왼쪽**으로 기울인 채 전진 — 뱅크 선회 (2026-09-16)
`bank_right`	같은 것을 **오른쪽**으로

`static`	**완전 정지** — 서보 3개 1500 고정, 추력 0. 구동 없이 센서만 측정. **`run_sec` 무시 — MANUAL 로 나올 때까지 계속** (2026-09-16, 10-01 지속형으로)

`turn`(헤딩 틀기)은 2026-09-14 요 제어 제거와 함께 없앴다 — 지정하면 모르는 시나리오로 거부된다.

완전 정지 (`static`, 2026-09-16). 구동을 완전히 끊고 압력·IMU 만 재는 시나리오다. 서보 3개가 1500 에, 꼬리가 1000 에 못박히고 **PID 가 통째로 멈춘다**.

- **왜 "추력 0, 롤·피치 0" 으로는 안 되나**: 목표가 0 이어도 로봇이 조금만 기울면 오차가 생기고 PID 가 그 오차만큼 지느러미를 움직인다. 실측(10° 기울인 상태): `straight` 는 서보가 1412 로 88 µs 움직였고, `static` 은 1500 그대로였다.
- 쓰임: 물속 **수심 드리프트**, 자이로 바이어스, 그리고 **대기창 물속 영점 검증** — 완전 정지이니 `Speed_wz`(54열)가 0 근처여야 맞다. 크게 나오면 영점이 잘못 잡힌 것이다.
- **길이는 사람이 정한다 (2026-10-01).** `run_sec` 을 보지 않고 **btn1 으로 MANUAL 에 나올 때까지** 계속 측정한다. 원하는 만큼 두었다가 btn1 을 짧게 누르면 끝난다 — 몇 초 잴지 미리 정해 파라미터를 바꿔가며 실험하는 게 실제 운용과 맞지 않아서 바꿨다(09-30 실기).

저널에 `시작! 'static' — btn1 으로 MANUAL 에 나올 때까지 계속 측정합니다` 가 뜨고, 2초마다 `%.1f초 경과` 가 찍힌다. **측정 구간은 CSV `State`(55열)가 2 인 구간**이다:

```
awk -F, '$55==2' fish_log_*.csv
```

- 구현: `/auto/command` 의 z 칸(요 제거로 비어 있던 자리)을 **서보 고정 플래그**로 쓴다. 1 이면 `pid_control_node` 가 PID 를 건너뛰고 `[1500,1500,1500,1000]` 을 낸다. 요 목표가 아니다.
- 기본 `combo_list` 에는 없다. 버튼으로 돌리려면 `ros2 param set /auto_scenario_node combo_list "straight,static"` 처럼 넣는다.

뱅크 선회 (`bank_*`, 2026-09-16). 요 서보는 그대로 1500 고정이고, 좌우 지느러미로 몸을 기울여 돈다(물고기·비행기의 방식). 롤 부호는 벤치 실측으로 확정: **롤 − = 왼쪽, 롤 + = 오른쪽** (좌로 기울여 −34.5°, 우로 +40.5°). 물속에서 확인할 것 두 가지 — ① 기울이면 실제로 휘는가, ② 어느 쪽으로 휘는가.

회전량은 CSV 7열 `Yaw` 로 잰다(**양수 = 좌선회**, 반시계): `State`(55열)가 2 인 구간의 끝 Yaw − 시작 Yaw. `bank_left` 인데 Yaw 가 음수로 가면 기운 반대쪽으로 도는 것이니 이름을 바꿔야 한다. 추력이 있어야 돈다 — `throttle_pct` 0 으로는 기울기만 하고 안 휜다.

`angle_deg` 는 지느러미 각이 아니라 목표 자세다. 지느러미는 *오차*에 비례해 꺾이고 (`kp` 8.75 → 피치 오차 1° = 혼 1°), 혼의 한계는 **±20°** 다. 그래서 수평에서 출발할 때:

angle_deg	출발 순간 지느러미	비고
15 (기본)	15°	가동범위의 75 %
20	**20° (포화)**	지느러미가 낼 수 있는 최대
30 이상	20° (포화)	더 안 꺾인다 — 포화가 **오래 지속**될 뿐이라 더 급해진다

로봇이 실제로 숙여지기 시작하면 오차가 줄어 지느러미도 중립으로 돌아온다. 즉 위 값은

출발 순간에만 나오는 최대치다. 물속에서 그 자세가 실제로 나오는지는 부력·중심·추력에

달렸고 **아직 확인 전이다** — 첫 시험은 15° 로 두고, 안 숙여지면 그때 올릴 것.

조종기로 전환 — btn1+btn2 함께 3초 (2026-09-03 실기 확인)

물가에서 노트북 없이 시나리오를 바꾸는 방법이다. 두 버튼을 **함께 3초** 유지하면

`combo_list`(**2026-10-01 현재 기본 `static` 하나** — 눌러도 바뀌지 않는다. 종전 기본은

`straight,dive,bank_left,bank_right` 로 직진·잠수·좌선회·우선회를 돌았다)를 한 칸 돈다.

```
btn1+btn2 함께 3초  →  시나리오 전환 (straight → dive → bank_left → bank_right → 처음).  **모드는 바뀌지 않는다**
btn1 단독           →  AUTO 진입 — 그때 걸려 있는 시나리오가 실행된다
```

3초를 채우는 순간 발화한다 — 손을 떼기 전에 이미 바뀐다(실측: 동시 시작 3.0초 뒤 전환).

3초에 못 미치면 아무 일도 일어나지 않지만, 저널에 몇 초를 눌렀는지 남는다.

조합으로 누르면 길이와 무관하게 **양쪽의 단독 동작이 전부 억제된다** — 모드 토글,

헤딩 영점, 지자기 보정, 대기압 재영점 어느 것도 발동하지 않는다. 손 떼는 시점이

어긋나도 안전하다(래치가 둘 다 뗀 뒤에 풀린다).

⚠ **지금 무엇이 걸려 있는지는 로봇을 봐서 알 수 없다.** 확인은 저널뿐이다:

```
[EKF] 두 버튼 동시 3초 — AUTO 시나리오 전환 신호를 보냈습니다.
[AUTO] 시나리오 전환 -> 'dive'  (2/4)  3.0초 추력 40% 각도 15.0도
[AUTO SCENARIO] AUTO 진입 — 10.0초 뒤 'dive' 시작  (3.0초, 추력 40%)
[AUTO SCENARIO] 시작! 'dive'  3.0초  추력 40%
```

3초를 못 채웠을 때는 이렇게 남는다:

```
[EKF] 두 버튼 조합이 3초에 못 미쳐(1.2초) 시나리오를 바꾸지 않았습니다.
```

목록은 바꿀 수 있다: `ros2 param set /auto_scenario_node combo_list "straight,dive,climb"`

터미널로 전환

```
ros2 param set /auto_scenario_node scenario dive    # straight/dive/climb/porpoise/attitude
ros2 param set /auto_scenario_node run_sec 5.0
ros2 param set /auto_scenario_node throttle_pct 30.0
ros2 param set /auto_scenario_node angle_deg 15.0
ros2 param set /auto_scenario_node start_delay_sec 10.0   # 진입 후 대기(초). 0.0 = 즉시
```

```
# 숫자는 소수점을 붙일 것 — 5 처럼 정수로 쓰면 형식이 달라 거부된다
# 그 다음 btn1 으로 MANUAL 로 나갔다가 AUTO 재진입
```

원리, 노드별로 먹히는지, 입력 형식 규칙은 **7장**.

바뀐 값은 다음 AUTO 진입부터 적용된다. 진입 순간에 대기 시간까지 설정이 통째로 잠기므로,

대기 중이나 달리는 중에 파라미터를 바꿔도 그 주행은 진입 때 값으로 끝까지 간다 — 사후에

"무엇을 지시했나"가 한 값으로 확정되게 하기 위해서다. 저널에는 이렇게 찍힌다:

```
[AUTO SCENARIO] AUTO 진입 — 10.0초 뒤 'dive' 시작  (5.0초, 추력 40%)
[AUTO SCENARIO] 대기 중 — 'dive' 시작까지 10초      ← 1초마다
[AUTO SCENARIO] 시작! 'dive'  5.0초  추력 40%
```

파라미터는 **재시작하면 사라진다**(기본값으로 복귀). 고정하려면 소스 기본값을 바꿀 것.

⚠ **진입 10초 뒤 추력이 걸린다** — 물 밖에서 실수로 누르면 잊을 즈음 꼬리가 3초간 40 %로 돈다.

대기 중에는 조용해서 AUTO 에 들어가 있다는 걸 놓치기 쉽다.

ESC 는 정상 동작이 확인된 부품이므로 **모터 전원이 붙어 있으면 정말로 돈다** —

규칙: 모터 전원을 연결할 때는 반드시 MANUAL 인지 먼저 확인할 것.

- **AUTO에서는 조종기가 꺼져도 멈추지 않는다** — 자율 주행의 정상 동작(설계 의도).

단 지금 시나리오는 시작 3초 뒤 스스로 추력을 끊으므로 폭주 위험은 그만큼 작다

- 나오기: btn1 짧게. **링크가 끊긴 AUTO에서 나오려면**:

`pkill -f state_estimation_ekf_node` (2초 뒤 respawn 되며 MANUAL로 복귀) 또는 서비스 정지

- AUTO 는 방위(요)를 지시하지 않는다 — 2026-09-14 요 제어 제거 이후 요 서보는 중앙 고정이다

4.4 링크 두절 (MANUAL)

nRF가 네 채널 `-9999`, 버튼 `255`, RSSI `-127`을 보낸다. PID는 즉시 중립

`[1500,1500,1500,1000]`으로 간다. **RSSI −127은 "약함"이 아니라 "전파 없음"이다** —

정상 연결은 −60~−80 dBm대다. 기동 직후에도 첫 정상 패킷 전까지는 항상 이 상태다.

---

5. 꺼내기 · 종료 · 보관

1. 🎮 **MANUAL인지 확인** (AUTO면 btn1 짧게), 스로틀 최하
2. 로봇을 꺼내고 물기를 닦는다
3. 💻 CSV를 회수할 거면 지금: `ls -t ~/ros2_ws/log_csv/fish_log_*.csv | head`

4. 💻 **로봇을 계속 켜둘 게 아니면 서비스를 내린다**:

```
sudo systemctl stop fish-robot
```

켜두면 **시간당 약 99 MB**씩 CSV가 쌓인다 (자동 삭제 없음 — 벤치 4일 방치로 7 GB 쌓인 전례)

5. 전원 차단

6. **압력 센서를 말린다** — MS5837-02BA 데이터시트는 대략 하루 1회 건조를 상정한다

7. 로그 정리 시 **`config/`는 절대 건드리지 않는다** (아래 표). `log_csv/`에서도 실험

CSV(`allan_`, `presschar_`, `tumble_`, `bench_` 접두사)는 남긴다 — 측정의 유일한 원본이다

`config/` 파일	**없으면**	**복구**
`mag_calib_params.txt`	방위가 무의미해진다 (조용히!)	`git checkout -- config/`
`port_map.txt`	포트 지문 대조가 죽는다 (조용히!)	`git checkout -- config/`
`hydro_zero.txt`	영점 재포착 필요 (btn1 3초면 됨)	자동 재생성

셋 다 없어도 WARN 한 줄 찍고 기본값으로 조용히 돈다 — 8/25에 지자기 보정 파일이

로그 정리에 휩쓸려 사라진 채 아무도 모르고 돌았던 사고가 이 표의 근거다.

---

6. 증상별 진단

증상	**원인 후보**	**확인 / 조치**
서보가 가만히 있는데 3초마다 움직인다	**AUTO 모드다** (시나리오 실행 중)	btn1 짧게. `/filtered/attitude`의 x가 ±5000 대면 AUTO
조종기가 안 붙는다	송신기 전원/바인딩/모델 선택	`/rc/status[4]` = **−127이면 전파 자체가 없음** (Pi·UART 문제 아님 — btn=255가 100 Hz로 오고 있으면 UART는 정상)
스틱을 미는데 지느러미가 찔끔	데드밴드(중립 ±1.5°)는 정상. 그 이상이면 게인	`ros2 param set /pid_control_node kp_pitch <값>` — 재빌드 불필요, 변경은 저널 WARN에 남는다
피치를 주면 롤이, 롤을 주면 피치가 움직인다	서보 장착 방향 반전 누락/과잉	`servo_reverse_*` 파라미터 토글 (3.4)
yaw가 이상하다	켤 때 자리 교란, 또는 지자기 보정	`dip_check.py` (D.3) — 판정대로 재시작 또는 보정
수심이 NaN	영점 미포착 / 정압 채널 사망	`/filtered/hydro` flags → `csv_format.md`의 진단 사다리
물속에서 btn1 3초가 안 먹는다	**정상** — 물속 대기압 영점 거부	물 밖에서 다시. 물속에서 필요한 건 `/hydro/zero_`

		q
속도가 NaN	자세 없음 / 정체압 사망 / q 영점 없음	flags 비트로 원인 특정 (csv_format.md)
물속 정지인데 Speed +0.3 m/s	공기 영점이 젖어서 흘렀다 (정상 현상)	Speed_wz(54열)를 쓴다. Zero_src(49열)가 2/0 이면 대기가 짧다 → start_delay_sec 10 (4.3)
공기 중 속도 −0.6 m/s	알려진 인공물 (4.2)	무시 — 물에서 사라짐
압력이 고정값으로 굳음	I2C 버스 잠김	[I2C Driver] ... 버스 잠김 의심 로그
자세가 안 나옴	nRF 무응답	timeout 5 cat /dev/ttyAMA0 | wc -c (정상 ≈ 23000/5초)
RPM이 0 또는 150 단위	정상	분해능이 150 RPM (1회전 = 4펄스, 100 ms 창)

벤치 스크립트를 쓸 때 — 발행자 충돌 주의

servo_direct.py 등은 /motor/output을 직접 발행한다. pid_control_node가 살아 있으면

두 발행자가 번갈아 밀어 서보가 떤다 → **서비스를 먼저 내릴 것.** rc_check.py/rc_map.py는

구독만 하므로 안전하다. (ros2 run 미등록 — python3 ~/ros2_ws/src/fish_robot_pkg/scripts/…)

스크립트별 용도·인자와 서비스를 내려야 하는지는 **부록 D**에 정리돼 있다.

유령 발행자 점검 — 측정 전 필수

```
ros2 topic info /filtered/attitude --verbose | grep -c "Endpoint type: PUBLISHER"   # 1
ros2 topic info /filtered/hydro --verbose | grep -c "Endpoint type: PUBLISHER"      # 1
ls -l /proc/*/exe 2>/dev/null | grep '(deleted)'
```

(deleted) 자체는 빌드 직후 정상이다. **유령의 표식은 ppid == 1**(고아 프로세스).

ros2 run을 백그라운드로 띄웠다 죽일 때는 래퍼가 아니라 **노드 프로세스를 직접** 죽여야

한다 — 래퍼 PID만 죽이면 노드가 고아로 살아남는다 (8/26에 6마리 쌓였던 실사례).

---

7. 파라미터 바꾸기 — 빌드·재시작 없이 (ros2 param)

게인, 서보 방향, AUTO 시나리오 값은 **서비스가 떠 있는 채로** 터미널에서 바꿀 수 있다.

소스를 고치면 빌드와 재시작이 둘 다 필요하지만(1.3), 이 방법은 둘 다 필요 없다.

7.1 원리 — 왜 빌드도 재시작도 필요 없나

1. 노드는 기동할 때 **소스에 적힌 기본값**으로 변수를 채운다 (declare_parameter).

2. 떠 있는 노드는 `/<노드 이름>/set_parameters` 서비스를 열어 두고 있다.

3. `ros2 param set` 은 이 서비스로 "값을 바꿔라"는 요청을 보낸다.

4. 노드 안의 **변경 콜백**이 요청을 받아 메모리의 변수에 새 값을 넣는다.

코드는 그대로이니 빌드가 필요 없고, 떠 있는 프로세스가 스스로 변수를 바꾸니 재시작도 필요 없다.

같은 이유로 **바꾼 값은 재시작하면 사라진다** — 메모리에만 있고 파일 어디에도 저장되지 않아서,

재시작하면 1번부터 다시 소스 기본값으로 채워진다.

7.2 ⚠ 노드마다 먹히는지가 다르다

4번의 콜백이 있는 노드만 실제로 바뀐다. **콜백이 없는 노드는 성공했다고 나와도 기동 때 읽은 값을 계속 쓴다.**

설정하면 `Set parameter successful` 이 나오고 `ros2 param get` 도 새 값을 보여 주기 때문에 속기 쉽다.

2026-09-15 격리 도메인 시험에서 `hydro_estimator_node` 의 `speed_k` 를 2.0 으로 "성공"시켰는데,

조회는 2.0 이었지만 변경 로그가 없었고 코드도 기동 때 한 번만 읽는다.

노드	실행 중 변경	반영 시점
`pid_control_node`	**된다**	**즉시** — 다음 자세 샘플(10 ms 안). 실측: 설정 직후 1 ms 안에 서보 출력이 바뀜
`auto_scenario_node`	**된다**	**다음 AUTO 진입**부터 — 진입 순간에 값을 잠근다 (4.3)
`state_estimation_ekf_node`	안 된다	소스 기본값 또는 런치 파일 수정 → 빌드 → 재시작
`hydro_estimator_node`	**`wet_zero_*` 만 된다** (2026-09-16)	**다음 AUTO 진입**부터 — 대기 시작 순간에 잠근다. 나머지(`speed_k` 등)는 여전히 안 된다
`uart_bridge` · `i2c_driver` · `rpm_driver` · `data_logger`	파라미터 없음	—

7.3 바꿀 수 있는 값

`pid_control_node` — 바꿀 때마다 저널에 WARN 이 남는다 (CSV 와 시각 대조용).

파라미터	형식	기본값	뜻
`kp_roll` / `ki_roll` / `kd_roll`	실수	8.75 / 0.0 / 0.0	롤 PID 게인
`kp_pitch` / `ki_pitch` / `kd_pitch`	실수	8.75 / 0.0 / 0.0	피치 PID 게인
`i_limit`	실수	300.0	적분 누산 상한
`servo_reverse_left` / `servo_reverse_right`	불리언	false / true	서보 하나만 반전 (3.4)

`invert_pitch` / `invert_roll`	불리언	false / false	축 하나만 부호 반전
`swap_rc_roll_pitch`	불리언	false	수동 스틱 롤↔피치 교환
`invert_rc_roll` / `invert_rc_pitch`	불리언	false / true	수동 스틱 부호 (AUTO 에는 안 걸림)
`rc_deadband` / `throttle_deadband`	정수	15 / 40	스틱 데드밴드 [카운트]

`ki_*` 를 바꾸면 그 축의 적분 누산이 0 으로 리셋된다. I·D 게인은 dt 를 게인에 흡수한 구현이라

100 Hz 기준 값이다 (CLAUDE.md §5). 요 게인·요 서보 반전은 2026-09-14 요 제어 제거로 없다.

`auto_scenario_node` — 다음 AUTO 진입부터 적용된다.

파라미터	형식	기본값	뜻
`scenario`	문자열	**static**	straight / dive / climb / porpoise / attitude / bank_left / bank_right / static
`start_delay_sec`	실수	10.0	AUTO 진입 후 대기(초). 0.0 = 즉시 출발. 물속 영점을 잡을 시간이다
`run_sec`	실수	3.0	주행 시간(초). 대기가 끝난 순간부터 센다. **`static` 은 이 값을 무시한다**
`throttle_pct`	실수	40.0	추력 % (0~100). attitude 는 무시하고 0
`angle_deg`	실수	15.0	dive/climb 의 목표 피치(도). 나머지 시나리오는 안 씀
`combo_list`	문자열	**static**	btn1+btn2 3초가 돌아가며 고르는 목록. 하나뿐이라 지금은 전환이 없다

> **2026-10-01: 기본값이 `static` 하나로 묶여 있다.** 운용자 요청으로 AUTO 진입이 무조건 완전 정지
> 측정이 되게 해 둔 상태이고, **시나리오 코드는 전부 그대로 남아 있다.** 주행 시험으로 돌아가려면
> 두 줄이면 된다(재시작하면 기본값으로 되돌아간다 — 영구히 바꾸려면 소스 기본값 수정):
> ```bash
> ros2 param set /auto_scenario_node scenario straight
> ros2 param set /auto_scenario_node combo_list "straight,dive,bank_left,bank_right"
> ```

`hydro_estimator_node` — `wet_zero_*` 만 실행 중 바뀌고, 다음 AUTO 진입(대기 시작)부터 적용된다.

나머지 파라미터(`speed_k`, `lever_x_m`, `atm_warmup_sec` …)는 "성공"이 떠도 안 먹힌다 (7.2).

파라미터	형식	기본값	뜻
`wet_zero_enable`	불리언	true	대기창 물속 영점 켜기/끄기
`wet_zero_still_depth_rate_mm_s`	실수	5.0	정지 판정: 수심 변화율 상한 [mm/s] (1초 선형회귀 기울기)
`wet_zero_still_gyro_dps`	실수	5.0	정지 판정: 원시 자이로 크기 1초 평균 상한 [도/초]. 폴백이 잦으면 7.0

`wet_zero_still_min_sec`	실수	2.0	정지가 이어져야 하는 최소 시간 [초]
`wet_zero_fallback_sec`	실수	2.0	정지 구간이 없을 때 쓰는 대기 마지막 구간 [초]
`wet_zero_min_samples`	정수	5	폴백에 필요한 최소 샘플 수. 미만이면 영점 없음(0)
`wet_zero_window_sec`	실수	1.0	변화율·자이로 평균 창 [초], 0.2 이상

7.4 명령과 입력 규칙

```
ros2 param list /pid_control_node                # 이름 목록
ros2 param get  /pid_control_node kp_pitch       # 지금 값
ros2 param set  /pid_control_node kp_pitch 10.0  # 바꾸기
ros2 param dump /auto_scenario_node              # 노드의 전체 값을 한 번에
```

형식이 다르면 거부되고 기존 값이 그대로 남는다 (2026-09-15 실측):

형식	받는 입력	거부되는 입력
실수	`5.0`, `10.0`	`5` — 정수로 읽힌다
정수	`20`	`15.0`
불리언	`true`, `false`	`1`, `0`
문자열	`dive`, `"straight,dive,climb"`	—

- **값의 뜻이 틀려도 거부된다**: 모르는 시나리오 이름, 음수 `start_delay_sec`, 빈 `combo_list`.
- `Node not found` 는 노드가 안 떠 있다는 뜻이다. 서비스 기동 직후라면 몇 초 뒤 다시.
- `ros2 param set` 명령 자체가 뜨는 데 1~2초 걸린다 (실측 1.6초). 반영은 명령이 끝나는 즉시다.

7.5 정한 값을 계속 쓰려면

런타임 값은 재시작하면 사라지므로, 확정한 값은 **소스의 기본값**으로 옮긴다.

이 저장소는 파라미터 파일(YAML)을 읽지 않는다 — 소스가 기본값의 유일한 보관처다.

단 EKF 는 런치 파일에서 넘기는 값(`mag_yaw_sigma` 등)이 소스 기본값보다 우선한다.

1. 해당 노드 소스의 `declare_parameter` 기본값을 고친다 (예: `auto_scenario_node.cpp` 파라미터 블록)
2. `cd ~/ros2_ws && colcon build --packages-select fish_robot_pkg`
3. `sudo systemctl restart fish-robot` — **빌드만 하면 도는 노드는 옛 값 그대로다** (1.3)
4. 기동 로그로 확인: `journalctl -u fish-robot -b | grep "Auto Scenario\] 시나리오"`
5. `CLAUDE.md` 와 이 문서에 적힌 기본값도 같이 고친다

부록

A. 명령어 모음

```
# 서비스
sudo systemctl {status,start,stop,restart} fish-robot
journalctl -u fish-robot -f                # 실시간 (노드 출력은 여기로 온다)
journalctl -u fish-robot -b | head -80     # 기동 로그

# 상태
ros2 topic echo /filtered/attitude --once   # roll(±5000이면 AUTO), pitch, yaw
ros2 topic echo /filtered/hydro --once      # [수심, 속도, q, q_raw, dP_hydro, 정압, pitch, flags, + 대기창 영점 7칸]
ros2 topic echo /auto/state --once          # 0 IDLE / 1 WAIT(진입 대기) / 2 RUN
ros2 topic echo /rc/status --once           # [btn1, btn2, V1×100, V2×100, RSSI]
ros2 topic echo /filtered/ekf_status --once # [10] = 플래그 (7 정상)

# 서비스 호출
ros2 service call /calibrate_mag std_srvs/srv/Trigger   # 지자기 보정 (재호출 = 조기 종료)
ros2 service call /hydro/zero_atm std_srvs/srv/Trigger  # 대기압 재영점 (물속 게이트 우회)
ros2 service call /hydro/zero_q  std_srvs/srv/Trigger   # 물속 q 영점

# 런타임 파라미터 (빌드·재시작 불필요, 재시작하면 소실 — 7장)
#   pid 는 즉시, auto_scenario·hydro(wet_zero_* 만) 는 다음 AUTO 진입부터. EKF 와 hydro 의 나머지는 "성공"이 떠도 안 먹힌다
ros2 param set /pid_control_node kp_pitch 10.0
ros2 param set /pid_control_node servo_reverse_right false
ros2 param get /pid_control_node kp_pitch
ros2 param set /auto_scenario_node start_delay_sec 10.0
ros2 param set /hydro_estimator_node wet_zero_still_gyro_dps 7.0   # 대기창 정지 판정 완화
ros2 param dump /auto_scenario_node

# 빌드 — 반드시 워크스페이스 루트에서 (src/ 안에서 돌리면 조용히 아무것도 안 된다)
cd ~/ros2_ws && colcon build --packages-select fish_robot_pkg
```

B. 정상값 요약

항목	정상값
`EKF_Flags` (`ekf_status[10]`)	**7** (9축) · 35 = 지자기 두절 6축 폴백
`hydro flags`	**0x01F** (육상·물속 공통; 공기 중 Speed 값은 무시)
`/rc/status` RSSI	**−60 ~ −80 dBm** (−127 = 링크 없음)
배터리 (실측 예)	V1 ≈ 23 V대, V2 ≈ 7.5 V대 — ÷100 해서 읽는다
`/filtered/attitude` 주기	100 Hz · `/sensor/pressure_raw` ~19 Hz · `/filtered/hydro` ~19 Hz (2026-09-19, 종전 11 Hz) · `/raw/imu_mpu9250` 100 Hz
`MPU_T` (2번째 IMU 칩 온도)	**~38 °C** 벤치 (CPU 열). `MPU_*` 가 전부 nan 이면 J4 확인 후 재시작 (감시 전용 — 제어

	무관). `MPU_A*`/`MPU_G*` 는 nRF IMU 축으로 회전돼 있어 `AX..GZ` 와 열끼리 비교

C. 로그가 남는 곳

종류	위치	자동 정리
텍스트 로그	journald (`journalctl -u fish-robot`)	있음 (디스크 10 % 한도)
ROS2 사본	`~/.ros/log/`	없음 — journald와 중복, 지워도 무방
데이터 CSV	`~/ros2_ws/log_csv/`	200 MB 회전만, **삭제 안 함** (시간당 ~99 MB)
보정·설정	`~/ros2_ws/config/`	**절대 지우지 말 것** (5장 표)

D. 벤치 스크립트 (`src/fish_robot_pkg/scripts/`)

대부분 `ros2 run` 에 등록돼 있지 않다 — **`python3 <경로>` 로 직접** 돌린다.

(`magneto_cal.py` 와 `verify_spin.py` 둘만 `ros2 run` 으로도 된다.)

```
# 매번 먼저 (모든 스크립트 공통)
cd ~/ros2_ws && source install/setup.bash

# "서비스 내릴 것" 표시가 있는 스크립트만
sudo systemctl stop fish-robot
#   ... 시험 ...
sudo systemctl start fish-robot      # 끝나면 반드시 되돌린다
```

아래에서 경로는 전부 `S=~/ros2_ws/src/fish_robot_pkg/scripts` 로 줄여 적는다.

종료는 따로 적지 않은 한 **Ctrl-C**.

D.1 조종기 · 서보

`rc_map.py` — 서비스 켠 채로

```
python3 $S/rc_map.py
```

스틱 4칸이 어느 값으로 오는지 막대로 본다. 채널 뒤바뀜(다른 칸이 움직임)과 부호 반전

(같은 칸, 반대 방향)을 가른다. Ctrl-C 하면 채널별 실측 범위를 찍는다.

`rc_check.py` — 서비스 켠 채로

```
python3 $S/rc_check.py
```

조종기→서보 경로 어디가 끊겼는지 좁힌다. `/rc/command` 가 안 변하면 조종기·수신기·nRF 쪽,

변하는데 `/motor/output` 이 안 따라오면 Pi 쪽.

`servo_toggle.py` — ⚠ **서비스 내릴 것**

```
sudo systemctl stop fish-robot
python3 $S/servo_toggle.py
```

2초마다 1350↔1650 을 번갈아 보낸다. 서보가 그 리듬으로 크게 움직이면 Pi→nRF→서보 경로가

살아 있는 것. 느리고 크게 하는 이유는 **떨림과 구분**하기 위해서다.

`servo_direct.py` — ⚠ **서비스 내릴 것**

```
sudo systemctl stop fish-robot
python3 $S/servo_direct.py
```

스틱을 서보에 직결한다(PID 없음). 책상 위에서는 로봇이 실제로 기울 수 없어 PID 가 포화되므로,

배선·방향·가동범위 확인은 이걸로 한다. **서보 중립(1500) 확인도 여기서.**

`servo_hold.py` — ⚠ **서비스 내릴 것**

```
sudo systemctl stop fish-robot
python3 $S/servo_hold.py            # 서보 1500, 꼬리 1000(정지)
python3 $S/servo_hold.py 1400       # 서보 값 지정
python3 $S/servo_hold.py 1500 1200  # 서보, 꼬리 ESC 값 지정
```

완전히 고정된 값을 100 Hz 로 계속 보낸다. 그래도 떨리면 명령과 무관(전원·PWM·서보 자체),

안 떨리면 명령의 1 µs 변동에 서보가 반응하는 것.

> `servo_*` 세 개는 `/motor/output` 을 직접 발행한다. 서비스를 켠 채로 돌리면
>
> `pid_control_node` 와 같은 토픽에 두 발행자가 붙어 명령이 두 값 사이에서 튄다.

D.2 압력

`mux_probe.py` — ⚠ **서비스 내릴 것** (I2C 버스를 직접 잡는다)

```
sudo systemctl stop fish-robot
python3 $S/mux_probe.py             # 채널당 5회 두드림
python3 $S/mux_probe.py 20          # 접촉 불량 의심이면 20회
```

네 단계를 **분리**해 진단한다 — ① 채널 전기 상태 ② 센서 응답·PROM(CRC) ③ **ADC 변환**

④ **배정 대조**. 앞 단계를 통과해도 뒤에서 떨어질 수 있고, "정상"은 실제 압력값(mbar)까지

뽑아낸 채널에만 붙는다. 탐지는 반드시 쓰기(리셋 0x1E)로 한다 — `i2cdetect` 로는 멀쩡한

센서도 못 찾는다.

출력 예 (2026-09-04 실측):

```
배정 파일: 물리 확인 완료(verified: yes) — 아래 지문이 이 배정의 근거입니다

ch0 (J5)  정상  정상      **front 자리 개체 교체됨** (파일 C1=44269 -> 실제 C1=41899), 1010.16 mbar 23.7°C, ADC 5/5
ch1 (J6)  정상  센서 없음  미연결이거나 완전 불량
ch2 (J7)  정상  ADC 사망  **right 자리 개체 교체됨** (파일 C1=41899 -> 실제 C1=43114), 변환명령 5/5 거부 — 개체 불량
```

④는 `config/port_map.txt` 의 지문(C1~C6 전부)과 지금 꽂힌 개체를 **도구가 대조해** 준다 —

숫자를 눈으로 맞출 필요가 없다. 센서를 바꿔 끼우는 선별 작업에서 제일 헷갈리는 지점이라

2026-09-04 에 넣었다. `verified:` 상태도 같이 알려 준다.

> **지문이 맞아도 그건 "그 개체가 그 자리에 있다"는 뜻일 뿐, 튜브가 어디로 가는지는 모른다.**
>
> 앞/좌/우 확정은 **바람 시험**뿐이다. 교체했으면 `rm ~/ros2_ws/config/port_map.txt` 후
>
> 한 번 띄워 재생성(`verified: no` 로 적힌다)하고, 바람 시험을 다시 한 뒤 yes 로 바꾼다.

`press_watch.sh` — ⚠ **서비스 내릴 것**

```
sudo systemctl stop fish-robot
bash $S/press_watch.sh
```

채널 접촉 상태 실시간 감시. 켜둔 채 커넥터를 눌러보거나 케이블을 흔들면 붙는 순간 ● 로

바뀐다 — 접촉 불량 자세를 찾는 용도.

`press_char.py` — 기록은 서비스 켠 채로, 분석은 아무 때나

```
python3 $S/press_char.py 3600                  # 기록 (초, 기본 3600)
python3 $S/press_char.py --s1 <csv>            # 안정화 시간
python3 $S/press_char.py --s2 <csv>            # 영점·현행 절차 오차
python3 $S/press_char.py --s3 <csv>            # 잡음·최대최소폭
python3 $S/press_char.py --s3 <csv> --after 300 # 안정 구간 시작점 지정
```

> **`ADC 사망` 이 뜨면** — 리셋도 받고 PROM 도 CRC 까지 맞는데 변환 명령(0x40~0x4A)만
>
> 전 OSR 에서 거부하는 고장이다. 2026-09-02(C1=42722)·09-04(C1=43114) 두 개체에서 나왔다.
>
> **저널에는 흔적이 남지 않는다** — `i2c_driver_node` 의 실패 ERROR 는 *전 채널*이 죽어야 뜨므로
>
> (`press_fail_streak_`), 셋 중 하나만 이러면 압력만 조용히 NaN 이 된다.
>
> 센서 불량인지 커넥터 문제인지는 **정상 채널과 서로 바꿔 꽂고 다시 재면** 갈린다 —
>
> 고장이 커넥터를 따라가면 배선, 개체를 따라가면 그 센서를 버린다.
>
> (이 3단계 검사는 원래 `mux_probe.py` 에 없어서 "정상" 오진을 냈다. 2026-09-04 에 합쳤다.)

D.3 자세 · IMU · 지자기

`magneto_cal.py` — 서비스 켠 채로

```
python3 $S/magneto_cal.py               # 보정 후 저장
python3 $S/magneto_cal.py --no-write    # 저장 없이 결과만 (먼저 이걸 권장)
ros2 run fish_robot_pkg magneto_cal.py  # 같은 것
```

여섯 자세를 하나씩 잡고 **그 자세 그대로 제자리에서 한 바퀴** 돈다. 진행 막대는 **자이로**로

잰다(자기장으로 재면 하드아이언 때문에 안 찬다). 흩어짐 8 % 넘으면 저장을 거부한다.

btn2 3초·`/calibrate_mag` 도 같은 스크립트를 돌리지만, 그쪽은 출력이 저널로 가서

막대 화면을 보려면 위 명령으로 직접 띄워야 한다.

`dip_check.py` — 서비스 켠 채로, **실험 직전에** (2026-09-15 신설)

```
python3 $S/dip_check.py             # 5초 측정. 로봇은 가만히
python3 $S/dip_check.py --sec 10
```

```
python3 $S/dip_check.py --no-journal # 저널 없이 지금 자리만
```

EKF 는 **서비스가 켜지는 순간 첫 지자기 샘플의 복각을 기준으로 잡고**, 그 뒤로 기준에서 ±15° 넘게 벗어난 샘플을 버린다. 켤 때 옆에 쇠붙이·자석이 있으면 기준이 틀리게 잡혀서, 깨끗한 곳으로 옮겨도 **정상 값이 전부 버려진다** (2026-09-14 20:17 실사례: 기준 175.8°, 버려진 값의 중앙값 144.3°). 저널에 ERROR 한 줄만 남기고 조용히 진행되므로 이 스크립트로 확인한다.

세 가지를 보고 판정한다. 토픽을 읽기만 해서 서비스와 충돌하지 않는다.

줄	보는 것	정상
① 기동 기준	이번 기동에서 EKF 가 저널에 남긴 복각 기준	143° ±10°
② 지금 자리	원시 지자기·가속도로 직접 잰 복각과 자기장 세기	143° ±10°, 세기 약 38 μT (±25 %)
③ EKF 사용	`/filtered/ekf_status[10]` 에서 나침판이 반영된 비율	50 % 이상

판정	뜻	할 일
✅ 실험 진행 가능	기준도 맞고 자리도 깨끗하다	그대로 진행
❌ 기준이 교란된 자리에서 잡힘	켤 때 자리가 나빴다. 지금 자리는 깨끗하다	**그 자리에서** `sudo systemctl restart fish-robot`
❌ 지금 자리가 교란됨	기준은 맞다	쇠붙이·자석에서 1 m 떨어진 곳으로 옮겨 다시 점검
❌ 자리도 기준도 안 맞음		깨끗한 곳으로 옮겨 점검 → 재시작 → 그래도 안 되면 보정(3.2)
⚠ 기준·자리는 정상인데 EKF 가 버림	방금 로봇을 움직였거나, **자세마다 복각이 달라지는 보정 문제**	가만히 두고 30초 뒤 재점검. 계속 버리면 다른 자세로 놓고 점검해 자세 사이 차이가 15° 넘으면 보정(3.2)
❌ 데이터 부족	IMU·지자기 중 무엇이 빠졌는지 개수로 알려준다	IMU 없음 = nRF 전원·UART 선. 지자기 없음 = J4 재장착 후 재시작

종료 코드: 0 = 진행 가능, 1 = 조치 필요, 2 = 데이터 없음. 계산은 EKF 와 같은 보정·축 변환을 쓴다 — 2026-09-14 기록 재생 시험에서 22:00 기동 142.1° / 38.2 μT 를 140.5° / 38.1 μT, 20:17 기동 175.8° / 67.9 μT 를 175.4° / 68.2 μT 로 재현했다.

출력 예 (기준이 틀리게 잡힌 경우):

```
① 기동 기준   175.8도
② 지금 자리  ✅ 복각 140.5 ± 1.1도 (편차 2.5도)   자기장 38.1 μT (평소 약 38)
             지금 복각과 기동 기준의 차이 35.3도 — EKF 복각 게이트에서 버려지는 차이 (±15도)
❌ 기준이 교란된 자리에서 잡혔습니다. 지금 자리는 깨끗하니 여기서 서비스를 재시작하세요:
```

`tilt6.py` — 서비스 켠 채로

```
python3 $S/tilt6.py
```

여섯 자세에서 기울기 정확도를 중력 벡터로 채점한다. 자세마다 놓고 Enter, 자세당 20초.

오일러각이 아니라 중력으로 채점하므로 피치 ±90° 근처도 유효하다.

`verify_spin.py` — 서비스 켠 채로

```
python3 $S/verify_spin.py
ros2 run fish_robot_pkg verify_spin.py   # 같은 것
```

360° 회전 끝단 검증. 자이로 적분각을 정답으로 놓고 지자기 헤딩이 되돌아옴 없이 360°까지

단조 증가하는지 본다.

`allan.py` — 기록은 서비스 켠 채로, 로봇을 **완전히 정지**시켜 둘 것

```
python3 $S/allan.py 7200          # 기록 (초, 2시간 이상 권장)
python3 $S/allan.py --an <csv>    # 분석
```

`magloss.py` — 서비스 켠 채로 (`i2c_driver_node` 를 SIGSTOP 으로 얼린다)

```
python3 $S/magloss.py
```

지자기 상실 → 6축 폴백 전환을 시험한다. 죽이지 않고 얼리는 이유는 죽이면 respawn 되고

압력까지 끊기기 때문. 전환 지연·튐·복귀를 본다.

`drift_test.py` — ⚠ **별도 기동 필요** (`use_mag:=false` 인 EKF 가 있어야 한다)

```
sudo systemctl stop fish-robot
# 터미널 1 — IMU 공급 + 지자기 끈 EKF
ros2 run fish_robot_pkg uart_bridge_node &
ros2 run fish_robot_pkg state_estimation_ekf_node --ros-args -p use_mag:=false
# 터미널 2
python3 $S/drift_test.py
```

6축 yaw 드리프트 게이트(≤ 2 °/분). launch 는 `use_mag` 를 True 로 박아두므로 서비스로는

이 시험을 못 한다 — 9축이면 지자기가 yaw 를 잡아줘 시험이 무의미하다.

`exp_b.py` — 옛 실험, **현재 무의미**

```
python3 $S/exp_b.py
```

> `exp_b.py` 는 Mahony 노드가 은퇴(2026-08)한 뒤로 `/filtered/attitude` 와
>
> `/filtered/attitude_ekf` 가 **둘 다 EKF** 라 같은 값을 비교한다. `tilt6.py` 도 같은 이유로
>
> 두 열이 소수 4째 자리까지 일치한다(왼쪽=제어 출력, 오른쪽=순수값). 남겨둔 것은 기록용이다.

E. 관련 문서

- `CLAUDE.md` — 설계 규약(±5000 인코딩, NaN 규약, 단위)과 하드웨어 특성
- `docs/csv_format.md` — CSV 62열 정의(48~55 대기창 영점, 56~62 MPU9250 2번째 IMU), Hydro 플래그 비트, 속도 NaN 진단 사다리

- `docs/imu_detailed.md` — 전 측정 이력 (§10 = 2026-08-25 압력 기동·지자기 재보정)
- `docs/todo.md` — 남은 작업 (ESC 아밍, 물속 검증, PID 물속 튜닝 등)

이 문서의 PDF판은 `docs/operation.pdf` — 갱신은 `python3 docs/make_pdf.py`.